

```
PlotlyBackend()
```

```
1 begin
2   using Pkg
3   Pkg.activate(".")
4   Pkg.instantiate()
5   using InfiniteOpt, Ipopt, Plots, NLSolve, LinearAlgebra
6   plotly()
7 end
```

Failed to load integration with PlotlyBase & PlotlyKaleido.
exception:

Error message from Main

```
ArgumentError: Package PlotlyBase not found in current path.
- Run `import Pkg; Pkg.add("PlotlyBase")` to install the PlotlyBase package.
```

```
Activating project at `~/.julia/pluto_notebooks`
```

```

1 begin
2   # Parameters (Sun-Earth 2D)
3    $\mu$  = 3.00348961491547e-6 # Sun = primary, Earth+Moon = secondary
4   const u_max = 0.1
5   const N_supports = 501
6   tf_lower = 50.0
7   tf_upper = 400.0
8   tf_start = 180.0
9   println("μ = ",  $\mu$ , ", u_max = ", u_max, ", supports = ", N_supports)
10
11  # Compute L1 (2D is identical)
12  function dOmega_dx(x,  $\mu$ )
13    r1 = abs(x +  $\mu$ )
14    r2 = abs(x - (1- $\mu$ ))
15    return x - (1- $\mu$ )*(x +  $\mu$ )/r1^3 -  $\mu$ *(x - (1- $\mu$ ))/r2^3
16  end
17  res = nlsolve(x -> dOmega_dx(x[1],  $\mu$ ), [0.99])
18  if !NLSolve.converged(res)
19    @warn "L1 solver did not converge"
20  end
21  x_L1 = res.zero[1]
22  println("Sun-Earth L1 at x = ", x_L1)
23
24  # Model + variables (2D only)
25  model = InfiniteModel(Ipopt.Optimizer)
26  @infinite_parameter(model,  $\tau$  ∈ [0, 1], num_supports = N_supports)
27  @variable(model, tf_lower <= tf <= tf_upper, start = tf_start)
28
29  # States (only x,y,vx,vy - z forced to zero)
30  @variable(model, x, Infinite( $\tau$ ), start =  $\tau$  -> (1- $\mu$  + 0.001)*(1- $\tau$ ) + x_L1* $\tau$ )
31  @variable(model, y, Infinite( $\tau$ ), start =  $\tau$  -> 0.0)
32  @variable(model, vx, Infinite( $\tau$ ), start =  $\tau$  -> 0.0)
33  @variable(model, vy, Infinite( $\tau$ ), start =  $\tau$  -> 0.0)
34
35  # Controls (only ux, uy - uz not needed)
36  @variable(model, -u_max <= ux <= u_max, Infinite( $\tau$ ))
37  @variable(model, -u_max <= uy <= u_max, Infinite( $\tau$ ))
38
39  # Potential gradients (2D versions)
40

```

```

41  r1(x_,y_) = sqrt((x_ + μ)^2 + y_^2)
42  r2(x_,y_) = sqrt((x_ - (1-μ))^2 + y_^2)
43  ∂Ωx(x_,y_) = x_ - (1-μ)*(x_ + μ)/r1(x_,y_)^3 - μ*(x_ - (1-μ))/r2(x_,y_)^3
44  ∂Ωy(x_,y_) = y_ - (1-μ)*y_/r1(x_,y_)^3 - μ*y_/r2(x_,y_)^3
45
46  model[:∂Ωx] = ∂Ωx
47  model[:∂Ωy] = ∂Ωy
48
49  # Dynamics + BCs + Objective (2D)
50  # Equations of motion
51  @constraint(model, deriv(x, τ) == tf * vx)
52  @constraint(model, deriv(y, τ) == tf * vy)
53  @constraint(model, deriv(vx, τ) == tf * (2*vy + model[:∂Ωx](x,y) + ux))
54  @constraint(model, deriv(vy, τ) == tf * (-2*vx + model[:∂Ωy](x,y) + uy))
55
56  # Initial: just outside Earth, zero velocity, in the x-axis
57  @constraint(model, x(0) == 1 - μ + 0.001)
58  @constraint(model, y(0) == 0.0)
59  @constraint(model, vx(0) == 0.0)
60  @constraint(model, vy(0) == 0.0)
61
62  # Final: exactly at L1, zero velocity
63  @constraint(model, x(1) == x_L1)
64  @constraint(model, y(1) == 0.0)
65  @constraint(model, vx(1) == 0.0)
66  @constraint(model, vy(1) == 0.0)
67
68  # Modified objective: minimum energy (tf * ∫ (ux^2 + uy^2) dτ)
69  @objective(model, Min, tf * ∫(ux^2 + uy^2, τ))
70
71  # Solve
72  set_silent(model)
73  @time optimize!(model)
74  println("Optimization done!")
75
76  # Extract solution
77  opt_x = value(x)
78  opt_y = value(y)
79  opt_vx = value(vx)
80  opt_vy = value(vy)

```

```

81     opt_ux = value(ux)
82     opt_uy = value(uy)
83     opt_tf = value(tf)
84      $\tau$ _grid = value.(supports( $\tau$ ))
85     t_grid =  $\tau$ _grid .* opt_tf
86
87     println("Transfer time = ", round(opt_tf, digits=2), " normalized units")
88     println("            $\approx$  ", round(opt_tf * 58.1328, digits=1), " days")
89     println("            $\approx$  ", round(opt_tf * 58.1328 / 365.25, digits=2), " years")
90
91     solution = (opt_x, opt_y, opt_vx, opt_vy, opt_ux, opt_uy, t_grid, opt_tf)
92
93     # Beautiful 2D plots
94     # Main trajectory
95     p_traj = plot(opt_x, opt_y, lw=2.5, label="Optimal low-thrust trajectory",
96                  title="Earth  $\rightarrow$  Sun-Earth L1 (planar CR3BP)", aspect_ratio=:equal,
97                  xlabel="x", ylabel="y", color=:purple)
98     scatter!(p_traj, [0.0], [0], marker=:circle, markersize=10, label="Sun")
99     scatter!(p_traj, [1- $\mu$ ], [0], marker=:star5, markersize=8, label="Earth")
100    scatter!(p_traj, [x_L1], [0], marker=:diamond, markersize=9, label="L1")
101
102    # Control history
103    p_ctrl = plot(t_grid, [opt_ux opt_uy], label=["ux" "uy"], lw=2,
104                  title="Thrust acceleration (normalized)", xlabel="Time")
105
106    # State history
107    p_states = plot(t_grid, [opt_x opt_y], label=["x" "y"], lw=2,
108                     title="Position vs time", xlabel="Time (normalized)")
109
110    display(p_traj)
111    display(p_ctrl)
112    display(p_states)

```

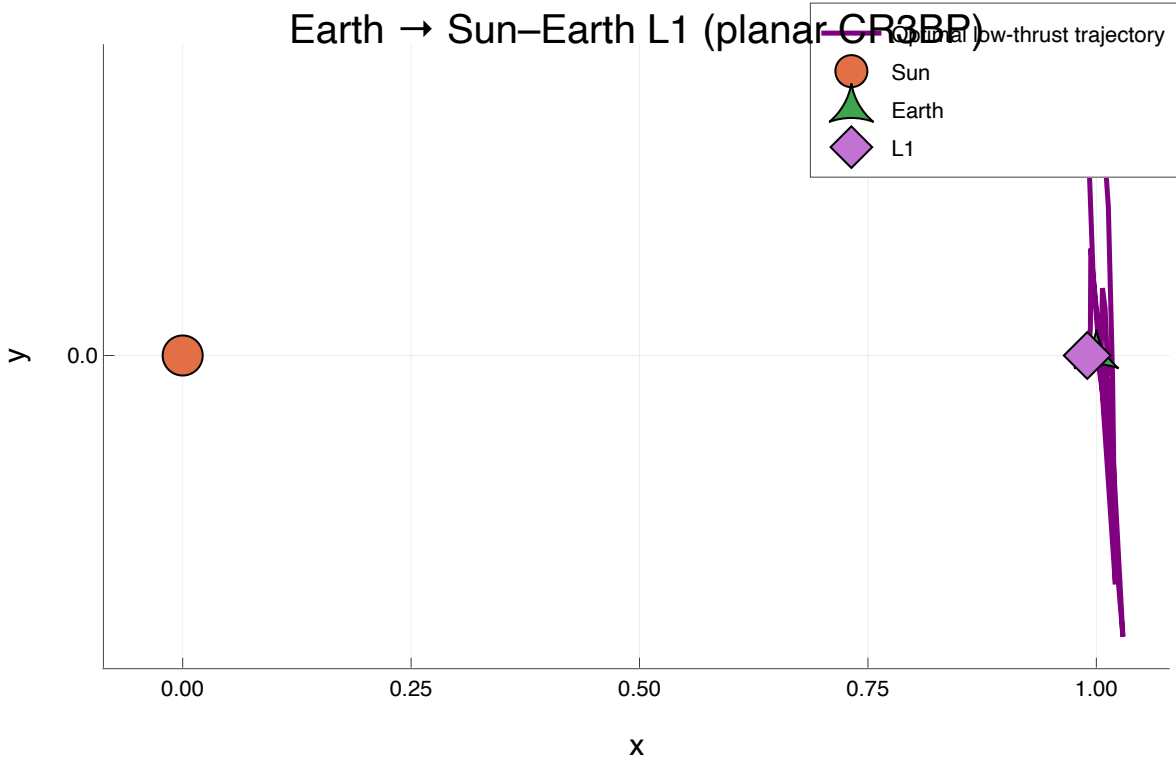
markershape star5 is unsupported with Plots.PlotlyBackend(). Choose from: [:auto, :circle, :cross, :diamond, :dtriangle, :hexagon, :hline, :none, :octagon, :pentagon, :rect, :utriangle, :vline, :x, :xcross]

```

 $\mu$  = 3.00348961491547e-6, u_max = 0.1, supports = 501
Sun-Earth L1 at x = 0.9900265839144706
9.492023 seconds (10.87 M allocations: 776.688 MiB, 8.95% gc time, 1.86% compilation time)

```

```
Optimization done!  
Transfer time = 400.0 normalized units  
              ≈ 23253.1 days  
              ≈ 63.66 years  
Plot{Plots.PlotlyBackend() n=4}  
Plot{Plots.PlotlyBackend() n=2}  
Plot{Plots.PlotlyBackend() n=2}
```



1 [p_traj](#)

